RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Course Code: CS1083 **Name of the Programme: B.Sc. Computer Science (Hons.)** **Name of the Course: Data Structures using C** **Semester: II**		
Course credit	**No. of hours per week (L + T + P)**	**Total no. of Teaching hours**
4	2+0+4	90
Course Objectives:		
a) Understand fundamental programming concepts essential for implementing data structures.		
b) Apply user-defined data types to implement linear data structures like linked lists, stacks, and queues.		
c) Implement binary trees and perform various traversal methods including inorder, preorder, and postorder.		
d) Analyze and apply tree-based data structures for efficient data storage and retrieval.		

Course outline (Syllabus of the course)

Syllabus:

Module - 1	**No. of hours**
	6
Foundations of Programming for data structures: Arrays, Pointers and Strings Functions and Recursion, Arrays, Pointers and Strings: Introduction to Arrays, Definition, One Dimensional Array and Multidimensional Arrays, Pointer, Pointer to Structure, various Programs for Array and Pointer. Strings. Introduction to Strings, Definition, Library Functions of Strings.	

Module - 2	**No. of hours**
	6
Implementing Data Structures with User-Defined Types: Linked Lists, Stacks, and Queues. Introduction to Data Structures, abstract data types, Stack ADT- definition and operations, Queue ADT- definition and operations, Linear list – Singly linked list implementation, insertion, deletion and searching operations on linear list.	

Module - 3	**No. of hours**
	6
Advanced Linked Lists: Implementation and Operations of Doubly and Circular Linked Circularly linked lists- Operations for Circularly linked lists, Doubly linked list implementation, insertion, deletion and searching operations, applications of linked lists	

Module - 4	**No. of hours**
	6
Binary Trees: Implementation and Traversal Techniques Trees – Definitions, tree representation, properties of trees, Binary tree, Binary tree representation, binary tree properties, binary tree traversals, binary tree implementation, applications of trees.	

Module - 5	**No. of hours**
	6
Graph Definition and properties of graphs, Types of graphs, Graph representations, Example of a basic graph implementation, Importance of graph traversal, BFS and DFS on Graph, Advantages and Disadvantages of Graph.	

Course outcomes
a) Understand the basic programming concepts needed to implement data structures.
b) Apply user-defined data types to implement linked lists, stacks, and queues.
c) Implement binary trees and perform different traversal methods like inorder, preorder, and postorder.
d) Analyze graph data structures, perform basic operations, and evaluate traversal techniques like BFS and DFS.

Text Books	
1	Karumanchi, N. (2016). Data structures and algorithms made easy (5th ed.). CareerMonk Publications. ISBN: 978-8193245279
2	Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1982). Data structures and algorithms (1st ed.). Addison-Wesley. ISBN: 978-0201000238

Reference books	
1	Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. ISBN: 978-0262046305
2	Sedgewick, R., & Wayne, K. (2011, with ongoing support). Algorithms (4th ed.). Addison-Wesley Professional. ISBN: 978-0321573513
3	Siddiqui, A. T., & Siddiqui, S. A. (2023). Data Structure Using C (1st ed.). CRC Press. ISBN: 978-1003453291

Reference links	
1.	https://www.geeksforgeeks.org/data-structures/

2.	https://www.scaler.com/topics/c-data-structures/

Lab Programs	**[60 Hrs]**
1	Write a program to find the largest and smallest element in an array.
2	Write a program to find the factorial of a number using recursion. Write a program to find the nth Fibonacci number using recursion.
3	Write a program to implement the Sudoku game structure and implement searching operations along rows, columns and grids.
4	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Implement the following operations: 1. insert_at_beginning(data): Insert a new node with the given data at the beginning of the list. 2. .insert_at_end(data): Insert a new node with the given data at the end of the list. 3. delete_node(data): Delete the first node in the list that contains the given data 4. traverse (): Traverse the list and print the data of each node
5	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Stack to represent the stack itself. Implement the following operations: 1. push(data): Push a new node with the given data onto the stack. 2. pop(): Remove and return the top node from the stack. If the stack is empty, return None. 3. peek(): Return the data of the top node without removing it. If the stack is empty, return None 4. empty(): Return True if the stack is empty, otherwise return False
6	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Queue to represent the queue itself. Implement the following operations: 1. enqueue(data): Add a new node with the given data to the end of the queue. 2. front(): Return the data of the front node without removing it. If the queue is empty, return None. 3. is_empty(): Return True if the queue is empty, otherwise return False
7	Write a program to create a class/structure DoublyNode to represent a node in a doubly linked list. Each node should have three attributes: data, next, and prev. Then, create a class/structure DoublyLinkedList to represent the linked list itself. Implement the following operations: 1. insert_at_beginning(data): Insert a new node with the given data at the beginning of the list. 2. insert_at_end(data): Insert a new node with the given data at the end of the list. 3. traverse_forward (): Traverse the list forward and print the data of each node.
8	Write a program to a class/structure CircularNode to represent a node in a circular linked list. Each node should have two attributes: data and next. Then, create a

	class/structure CircularLinkedList to represent the linked list itself. Implement the following operations: 1.insert_at_beginning(data): Insert a new node with the given data at the beginning of the list. 2. traverse(): Traverse the list and print the data of each node.
9	Write a program to create a class/structure TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself. Implement the following operations: 1. insert(data): Insert a new node with the given data into the binary tree (assume a simple insertion without balancing). 2. pre_order_traversal(): Perform a pre-order traversal of the tree and print the data of each node.
10	Write a program to create a class TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself. include a method/function find_lca(node1, node2) that takes two node values as input and returns the value of their lowest common ancestor. Assume all values in the tree are unique.
11	Write a program to create a class TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself, to include a method/function find_grandchildren(node) that takes a node value as input and returns a list of values of all the grandchildren of the given node.
12	Create a class/structure Graph to represent a graph using an adjacency list. Implement the following operations: 1. add_edge(v, w): Add an edge between vertices v and w.
13	Create a class/structure Graph to represent a graph using an adjacency list, to include a method/function dfs(start_vertex) that performs a Depth-First Search starting from start_vertex and prints the order of traversal.
14	Create a class/structure Graph to represent an undirected graph using an adjacency list. Include a method/function detect_cycle() that detects if there is a cycle in the graph. The method should return True if a cycle is detected, otherwise False.
15	A social network is represented as an unweighted graph where users are vertices and friendships are edges. Implement a class/structure SocialNetwork that supports the following operations: 1. add_friendship(user1, user2): Add a friendship (edge) between user1 and user2. 2. degrees_of_separation(user1, user2): Find the shortest path (in terms of the number of edges) between user1 and user2 using BFS and return the number of edges in the path.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Name of the Programme:	**BSc(H)**	
Title of the Course:	**Data Structures Using C**	
Course code	**CS1083**	
Semester:	**II**	
Names of the Course Instructor(s)	**Prof.Ashwini Prasad S** **Prof.Aruna S**	
Course credit	**No. of hours per week (L + T + P)**	**Total no. of Teaching hours**
4	**2+0+4**	**90**

Session	Module No.	Topic	Pedagogy /activities	Pre-reading/reference
1-2	**1**	Functions & Recursions,Arrays:Introduction to Arrays, Definition, One Dimensional Array and Multidimensional Arrays	Chalk & Talk, Flow Diagrams, Board Tracing, Live Coding, Think–Pair –Share, Error-based Learning, Dry run exercises	Horowitz & Sahani –Ch.2,Geeks for Geeks
3-4		Pointers:Pointer, Pointer to Structure, various Programs for Array and Pointer.	Memory diagrams, Examples, Index tracing, Instructor demonstration, Group activities, Row/column-major visualization Analogies, Memory maps, Live coding,	Horowitz & Sahani – Ch.1,Geeks for Geeks

			Pointer arithmetic demos, Debugging exercises	
5-6		Strings:Introduction to Strings, Definition, Library Functions of Strings.	Demonstrations of library functions, Writing custom string functions, Pair-tracing activities, Mini string-manipulation labs	Geeks for Geeks
7-18		**LAB:** 1. Write a program to find the largest and smallest element in an array. 2. Write a program to find the factorial of a number using recursion. Write a program to find the nth Fibonacci number using recursion. 3. Write a program to implement the Sudoku game structure and implement searching operations along rows, columns and grids.	Coding + Dry-run	Class Materials
19-20	**2**	Introduction to Data Structures, ADT concept	Concept discussion, analogies, diagrams, Q&A, brainstorming, quizzes	Narasimha–Ch.1
21-22		Stack ADT- definition and operations, Queue ADT- definition and operations	Visual explanation of push/pop, live coding,	Narasimha–Ch.4,5

			dry-run, overflow/underflow exercises Real-life analogy, pointer movement diagrams, paper tracing, class simulation	
23-24		Linear list – Singly linked list implementation, insertion, deletion and searching operations on linear list.	Node diagrams, memory maps, live coding, create & trace list operations, debugging	Narasimha–Ch.3
25-28		**LAB:** 5. Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Implement the following operations: 1. insert_at_beginning(data): Insert a new node with the given data at the beginning of the list. 2. .insert_at_end(data): Insert a new node with the given data at the end of the list. 3. delete_node(data): Delete the first node in the list that contains the given data 4. traverse (): Traverse the list and print the data of each node	Coding + Dry-run	Class Materials
29-32		**LAB:** 6. Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then,	Coding + Dry-run	Class Materials

		create a class/structure Stack to represent the stack itself. Implement the following operations: 1. push(data): Push a new node with the given data onto the stack. 2. pop(): Remove and return the top node from the stack. If the stack is empty, return None. 3. peek(): Return the data of the top node without removing it. If the stack is empty, return None 4. empty(): Return True if the stack is empty, otherwise return False		
33-36		**LAB:** 7. Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Queue to represent the queue itself. Implement the following operations: 1. enqueue(data): Add a new node with the given data to the end of the queue. 2. front(): Return the data of the front node without removing it. If the queue is empty, return None. 3. is_empty(): Return True if the queue is empty, otherwise return False	Coding + Dry-run	Class Materials
37-39	**3**	Circularly linked lists- Operations for Circularly linked lists**,**applications	Diagrams, comparisons, live coding, traversal walkthrough, physical simulation, lab tasks	Narasimha–Ch.3

40-42		Doubly linked list- Operations for Circularly linked lists**,**applications	Memory pointer visuals, code demos, tracing, debugging, applications	Narasimha–Ch.3
43-46		**LAB:** 8. Write a program to create a class/structure DoublyNode to represent a node in a doubly linked list. Each node should have three attributes: data, next, and prev. Then, create a class/structure DoublyLinkedList to represent the linked list itself. Implement the following operations: 1. insert_at_beginning(data): Insert a new node with the given data at the beginning of the list. 2. insert_at_end(data): Insert a new node with the given data at the end of the list. 3. traverse_forward (): Traverse the list forward and print the data of each node.	Coding + Dry-run	Class Materials
47-50		**LAB:** 9. Write a program to a class/structure CircularNode to represent a node in a circular linked list. Each node should have two attributes: data and next. Then, create a class/structure CircularLinkedList to represent the linked list itself. Implement the following operations: 1.insert_at_beginning(data): Insert a new node with the given data at the beginning of the list. 2. traverse(): Traverse the list and print the data of each node.	Coding + Dry-run	Class Materials

51-52	**4**	Trees – Definitions, tree representation, properties of trees,applications of trees	Examples (family tree, file system), diagrams, properties exercises, representation comparisons Case-based learning, real application demonstrations, mapping activities	Narasimha–Ch.6
53-54		Binary tree, Binary tree representation, binary tree properties	Diagram-based teaching, array vs linked representation, classification exercises	Narasimha–Ch.6
55-56		Binary tree traversals, binary tree implementation,	Board walkthroughs, recursion tracing, coding tasks, group tracing activities	Narasimha–Ch.6
57-60		**LAB:** 10. Write a program to create a class/structure TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself. Implement the following operations:	Coding + Dry-run	Class Materials

		1. insert(data): Insert a new node with the given data into the binary tree (assume a simple insertion without balancing). 2. pre_order_traversal(): Perform a pre-order traversal of the tree and print the data of each node.		
60-64		**LAB:** 11.Write a program to create a class TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself. include a method/function find_lca(node1, node2) that takes two node values as input and returns the value of their lowest common ancestor. Assume all values in the tree are unique.	Coding + Dry-run	Class Materials
65-68		**LAB:** 12.Write a program to create a class TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself, to include a method/function find_grandchildren(node) that takes a node value as input and returns a list of values of all the grandchildren of the given node.	Coding + Dry-run	Class Materials
69-71	**5**	Definition and properties of graphs, Types of graphs, Graph representations, Example of a basic graph implementation,Advantages and Disadvantages of Graph.	Diagrams, real-life examples, Q&A, drawing graphs Visual comparison , classificatio n exercises	Narasimha–Ch.9

			Chalk & talk, adjacency matrix/list creation, coding demo Code walkthrough, memory visualization, mini-lab Discussion, case studies	
72-74		Importance of graph traversal, BFS and DFS on Graph	Real-world analogies, exploration activity Diagram tracing, queue simulation, live coding Recursion & stack tracing, coding demo, cycle detection tasks	Narasimha–Ch.9
75-78		**LAB:** 13.Create a class/structure Graph to represent a graph using an adjacency list. Implement the following operations: 1. add_edge(v, w): Add an edge between vertices v and w.	Coding + Dry-run	Class Materials
79-82		**LAB:** 14.Create a class/structure Graph to represent a graph using an adjacency list, to include a method/function dfs(start_vertex) that performs a Depth-First Search starting from start_vertex and prints the order of traversal.	Coding + Dry-run	Class Materials

83-86		**LAB:** 1. Create a class/structure Graph to represent an undirected graph using an adjacency list. Include a method/function detect_cycle() that detects if there is a cycle in the graph. The method should return True if a cycle is detected, otherwise False.	Coding + Dry-run	Class Materials
87-90		**LAB:** 1. A social network is represented as an unweighted graph where users are vertices and friendships are edges. Implement a class/structure SocialNetwork that supports the following operations: 1. add_friendship(user1, user2): Add a friendship (edge) between user1 and user2. 2. degrees_of_separation(user1, user2): Find the shortest path (in terms of the number of edges) between user1 and user2 using BFS and return the number of edges in the path.	Coding + Dry-run	Class Materials

Assessment and Evaluation

Continuous Internal Evaluation (CIE)	
Components of CIE	Weightage of each component
CIE-1:Short Test	10 marks out of 70
CIE-1:Scenario-Based Test	10 marks out of 70
CIE 2:Theory	25 marks out of 70
CIE 3: Practical Test	20 marks out of 70

CIE 3: Lab Program Submissions+ Viva	5 marks out of 70

Semester End Exams (SEE)	
Mode of Exam	Practical
Weightage of exam	30

Course Outcomes- Programme Outcomes Matrix

Sl.No	Course outcome	Programme Outcome											
		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
1	CO1: Understand basic programming concepts for data structures	3	2	1	2	1	1	1	2	1	1	1	1
2	CO2: Apply user-defined data types to implement stacks, queues & linked lists	3	3	2	3	1	2	2	2	1	1	1	1
3	CO3: Implement binary trees and traversal methods	3	3	2	3	1	2	2	2	1	1	1	1
4	CO4: Analyze graph structures and	3	3	3	3	1	2	2	3	1	1	1	1

	traversal techniques (BFS/DFS)												

Rubrics for evaluation of CIE

Total Marks		**20**
CIE-1 Components		**Rubrics for Assessment**
CIE-1	Short Test Graded Component-1	Two marks of 5 questions will be conducted for 10 marks. Each right answer carries two marks. No negative marking for wrong answers.
	Scenario Based Test Graded Component-2	5 Questions for 10 marks,2 marks for each question.

Total Marks		**25**
CIE-2 Components		**Rubrics for Assessment**
CIE-2	Theory	The test will be conducted for 25 marks and will be evaluated according to the Scheme of Evaluation prepared by the team of course faculty.

Total Marks		**25**
CIE-3 Components		**Rubrics for Assessment**
CIE-3	Lab Program Submission + Viva-Voce	Lab Program + Viva-Voce = 5 marks ***(Refer Table-1)***

	Graded Component-1	
	Practical Test Graded Component-2	• Write-up:12 marks out of 20 (2 programs,each program carries 6 marks) • Execution:4 marks out of 20 (1 program execution) • Viva-Voce:4 marks out of 20 ***(Refer Table-2)*** Total:20

Note:Total Lab Program Submission + Viva-Voce: 15 programs each carries 10 marks which sums up to 150 marks and will be down scaled to 5 marks.

Total Marks		**30**
SEE Components		**Rubrics for Assessment**
SEE	Practical	Write-up:20 marks out of 30(2 programs) Execution: 5 marks out of 30(1 program execution) Viva-Voce: 5 marks out of 30 ***(Refer Table-3)***

Table-1
Practical Component Evaluation Rubrics: Graded Component 1
Lab Program Submission (GoogleClassRoom) + Viva (10 Marks)

Lab Program Submission (GoogleClassRoom):6Marks

Sl. No	Criteria	Measuring Method	Excellent	Very Good	Good	Average	Needs Improvement
1	Code Quality & Logic(3)	Review of code in Google Classroom submission	Code is efficient, well-structured, uses correct logic; no errors(3)	Minor issues but overall correct and efficient(2)	Logic partially correct; minor errors (1)	Code runs with multiple errors; logic unclear (0.5)	Major issues; code does not run (0)
2	Output Accuracy(2)	Program execution and screenshots	Accurate output for all test cases (2)	Minor deviations in output (2)	Correct for basic cases only (1)	Output partially correct (0.5)	Output incorrect (0)

3	Timely Submission(1)	Timestamp in Google Classroom	Submitted before deadline (1)	Submitted on deadline (0.8)	Submitted within 12 hours after deadline (0.6)	Submitted within 24 hours after deadline (0.4)	Not submitted (0)

Viva-Voce:4Marks

One mark questions will be conducted for 4 marks. Each right answer carries one mark. No negative marking for wrong answers.

Table-2
Practical Component Evaluation Rubrics: Graded Component 2

Sl. No	Criteria	Measuring Method	Excellent	Very Good	Good	Average	Needs Improvement
1	Program Write-up (per program 6M) (12M)	Review submitted document (PDF/notebook)	Complete and accurate problem statement, algorithm, and code; well-organized (6)	Minor omissions or formatting issues; logic mostly correct (5)	Partial write-up; minor errors in logic (3-4)	Weak write-up; incomplete algorithm/code (2)	Missing or incorrect write-up (1)
2	Program Execution & Output (1 program execution) (4M)	Instructor observes live execution or reviews recorded output	Code runs successfully; outputs correct results without guidance (4)	Minor errors; outputs mostly correct; little guidance required (3)	Runs with partial correctness; some guidance needed (2)	Runs with errors; significant guidance needed (1)	Code does not run(0)
3	Conceptual Understanding & Explanation (4M)	Oral questioning during viva	Clear, confident, complete understanding; can explain logic, algorithms, and output (4)	Good understanding; minor gaps; mostly clear explanation (3)	Basic understanding; partially correct explanation (2)	Weak understanding; explanations unclear (1)	Cannot explain; no understanding (0)

Table-3
Semester End Examinations

Sl. No	Criteria	Measuring Method	Excellent	Very Good	Good	Average	Needs Improvement
1	Program Write-up (per program 10 M) (20M)	Review submitted document (PDF/notebook)	Complete and accurate problem statement, algorithm, and code; well-organized (9-10)	Minor omissions or formatting issues; logic mostly correct (7-8)	Partial write-up; minor errors in logic (5-6)	Weak write-up; incomplete algorithm/code (3-4)	Missing or incorrect write-up (0-2)
2	Program Execution & Output (1 program execution) (5M)	Instructor observes live execution or reviews recorded output	Code runs successfully; outputs correct results without guidance (5)	Minor errors; outputs mostly correct; little guidance required (4)	Runs with partial correctness; some guidance needed (3-2)	Runs with errors; significant guidance needed (1)	Code does not run(0)
3	Conceptual Understanding & Explanation (5M)	Oral questioning during viva	Clear, confident, complete understanding; can explain logic, algorithms, and output (5)	Good understanding; minor gaps; mostly clear explanation (4)	Basic understanding; partially correct explanation (3-2)	Weak understanding; explanations unclear (1)	Cannot explain; no understanding (0)

Signature of the Course Faculty